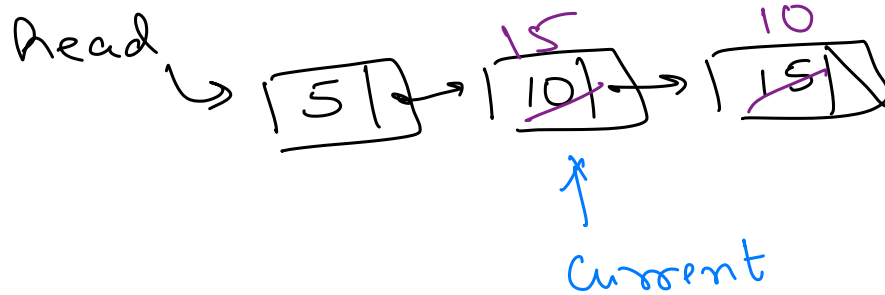


Delete a given node



```
deleteNode(  
    Node current  
)  
{  
:  
}
```

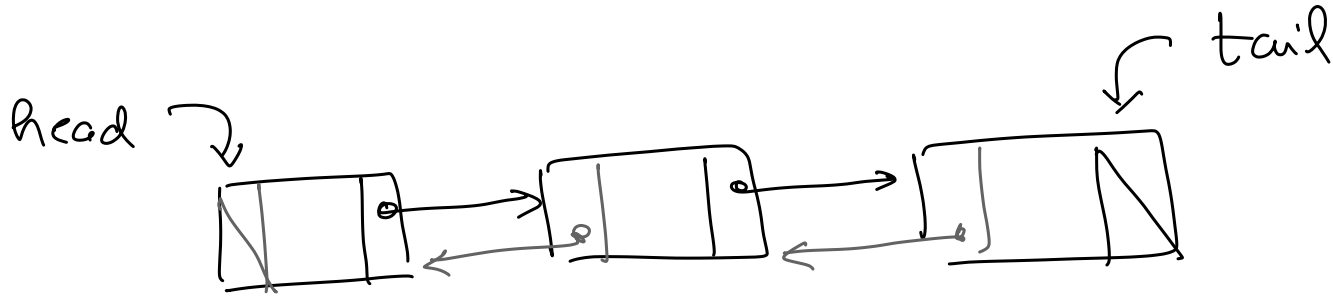
① Swap values of current & its next node.

② Delete current's next node.

Limitation: will not work if deleting last node.

Doubly Linked List

linked list in which each node keep track of its two adjacent nodes.



```
class Node {  
    int data;  
    Node next;  
    Node previous;  
};
```

Traversal $\rightarrow$ forward traversal $\leftarrow$ Same as singly list
 $\searrow$ backward traversal $\leftarrow$ start with last node and move backwards.

Forward Traversal

- Set current to first node of list. $\rightarrow$ Node current = head;
- while (current is not empty) do $\rightarrow$ while (current != null) {
 - Process current node. $\rightarrow$
 - Set current to current node's next. $\rightarrow$ current = current.next;
- Stop. $\}$

Backward Traversal

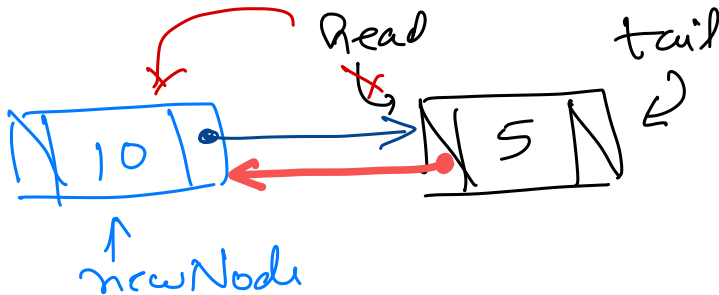
- Set current to last node of list. $\rightarrow$ Node current = tail;
- while (current is not empty) do $\rightarrow$ while (current != null) {
 - Process current node. $\rightarrow$
 - Set current to current node's previous. $\rightarrow$ current = current.previous;
- Stop. $\}$

AddAtFront(element)

- Make space for new element, say newNode.
- Store element in newNode's data.
- Set previous of newNode as empty.
- if list is empty then
 - Make newNode as last node.

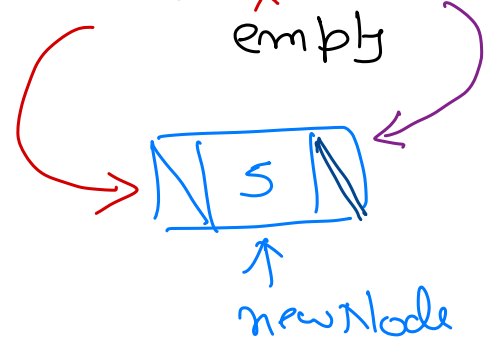
Else

- Set newNode as previous of head.
- Set newNode's next to head.
- Set head to newNode.
- Stop.

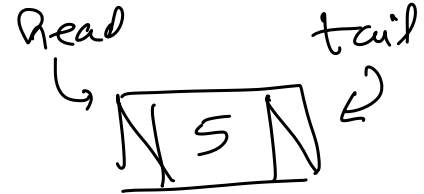


add AtFront (5)

Head ~~→~~ ~~→~~ tail
empty



⇓



AddAtEnd(element)

- Make space for new element, say newNode.
- Store element in newNode's data.
- Set newNode's next to empty.
- Set previous of newNode as empty.
- if list is empty then
 - Set head to newNode.
 - Set tail to newNode.
 - Stop.
- Set newNode as next of tail node.
- Set previous of newNode to tail.
- Set tail to newNode.
- Stop.

addAtEnd(5)

Head ~~→~~ empty ~~←~~ tail

tail.next = newNode



newNode

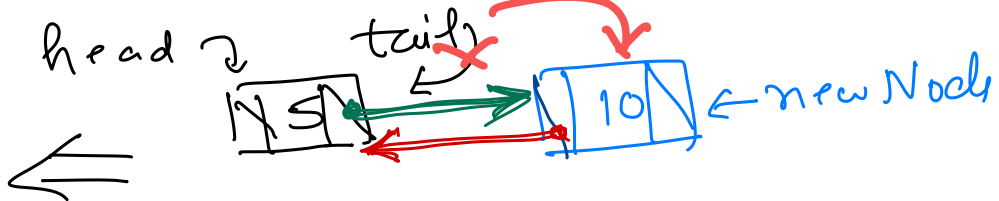
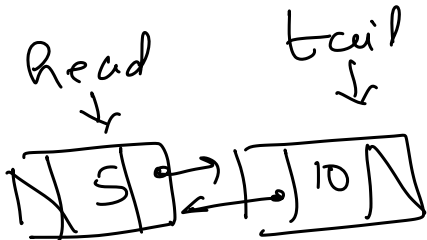
head

tail



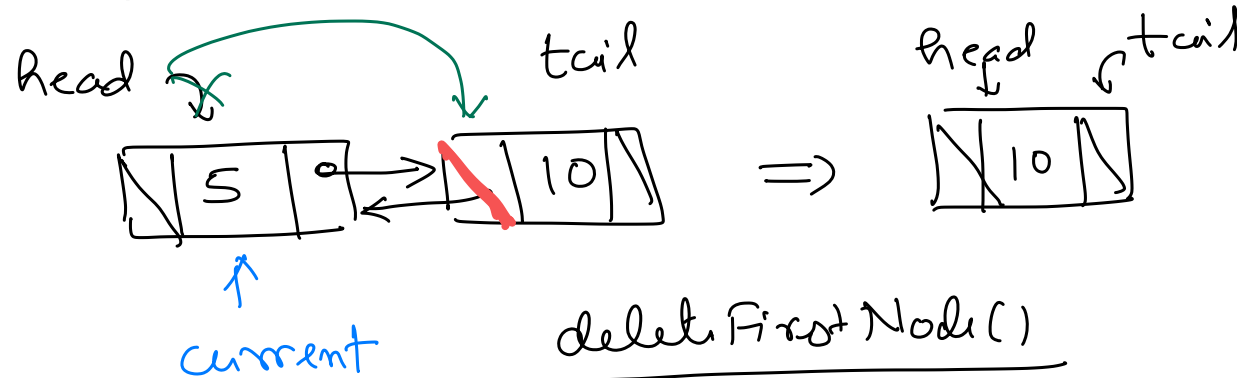
newNode.previous = tail

addAtEnd(10)

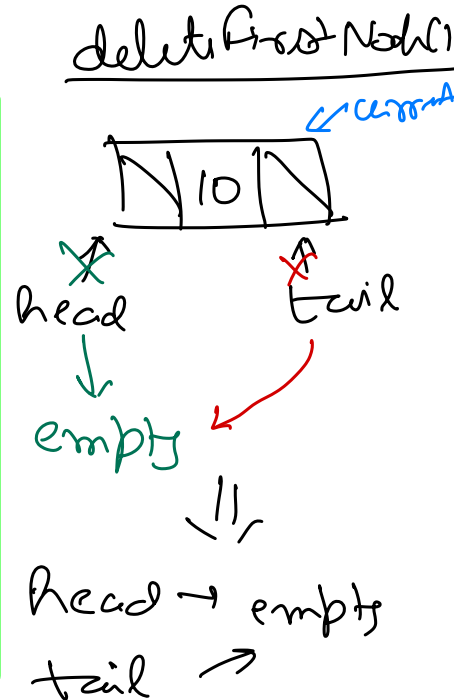


DeleteFirstNode()

- if list is empty then
 - Stop.
- Set current to head
- Move head to head's next node.
- if list is empty then // Check if head is empty, as head got changed
 - Set tail to empty
- Else // list is not empty
 - Set previous of head to empty
 - Release current node. > Not needed in JAVA.
 - Stop.

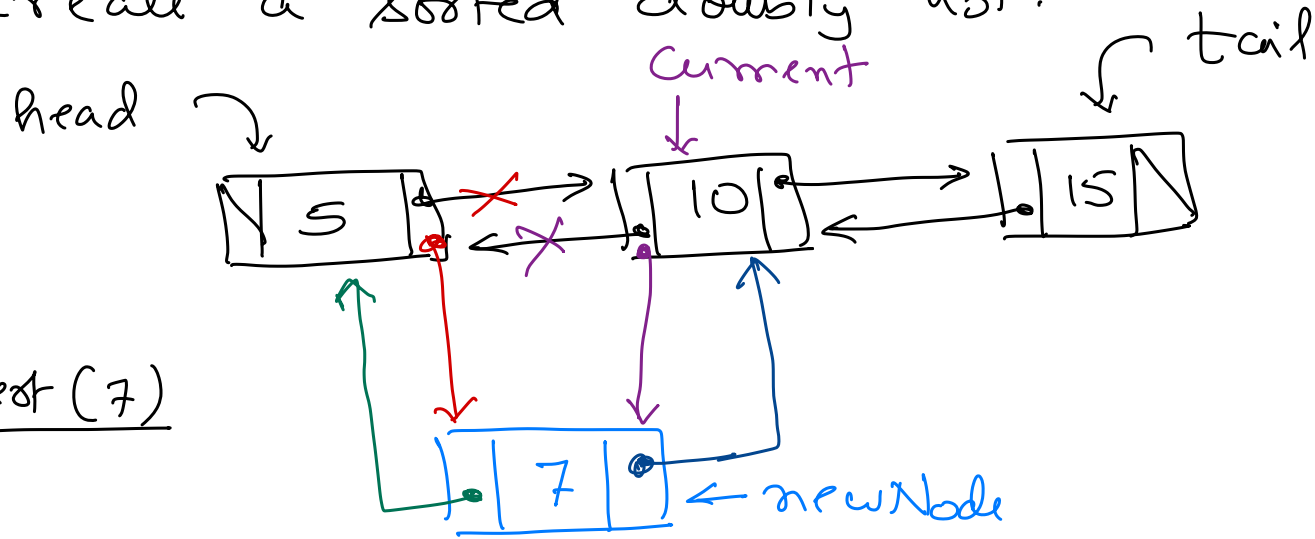


deleteFirstNode()



Insert in doubly list

→ Create a sorted doubly list.



insert(7)

a) Set new Node's next to current
 $\text{newNode.next} = \text{current};$

b) Set new Node's previous to current's previous.

$\text{newNode} \cdot \text{previous} = \text{current} \cdot \text{previous}$

c) Set current node's previous node's next to new Node.

$\text{current} \cdot \text{previous} \cdot \text{next} = \text{newNode}$

d) Set current's previous to newNode.

$\text{current} \cdot \text{previous} = \text{newNode}$

Special cases

① List is empty. $\Rightarrow$ Set newNode as first and last node.

② Adding smallest value. $\Rightarrow$ Add newNode as first node.

③ Adding largest value. $\Rightarrow$ Add new Node as last node.

Insert(element)

// 1. Create new node

- Make memory for new element, say newNode.
- Store element in newNode's data.
- Set newNode's next and previous to empty.

// 2. If list is empty?

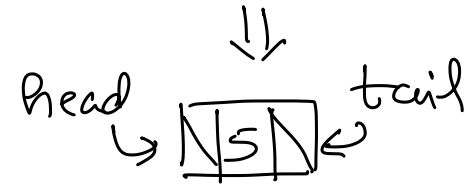
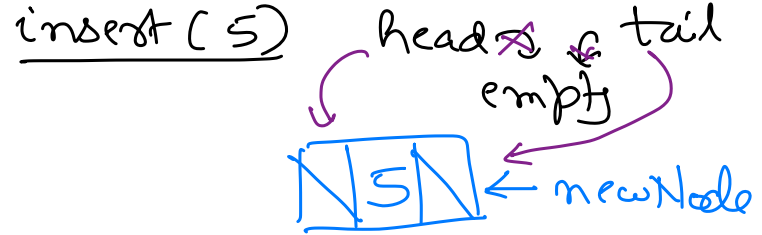
- if head is empty then

// Make newNode as the first and last node of the list.

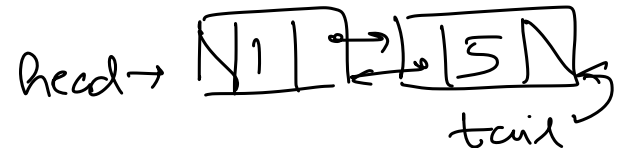
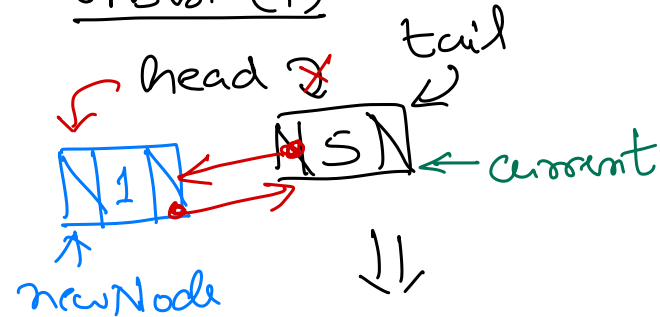
- Set head and tail to newNode.
- Stop.

// 3. Traverse list to find node - current node.

- Set current to head (first node).
- while (current is not empty) do
 - if (current node's data > element) then
 - // Found the node, end the traversal.
 - End the traversal.
 - Set current to current's next node.



insert (1)



// 4. If adding before the first node? - Current is the first node.

- if (current is head) then

- Before the first node comes newNode. // Set head's previous to newNode.
- After newNode comes the first node. // Set newNode's next to head.
- Make newNode as the first node. // Set head to newNode.
- Stop.

// 5. If adding after the last node? - Current is empty

- if (current is empty) then

- After the last node comes newNode. // Set tail's next to newNode.
- Before newNode comes the last node. // Set newNode's previous to tail.
- Make newNode as the last node. // Set tail to newNode.
- Stop.

// 6. Add a new node between current and current's previous node.

- Make the current node come after newNode. // Set newNode's next to current.

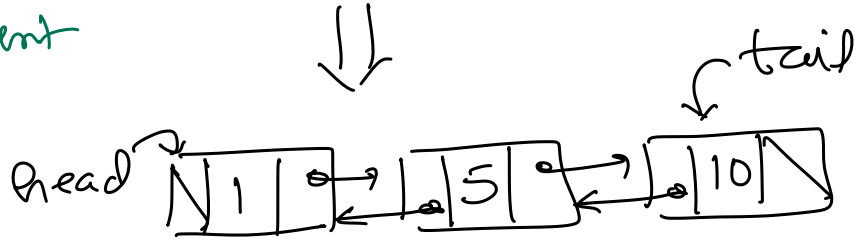
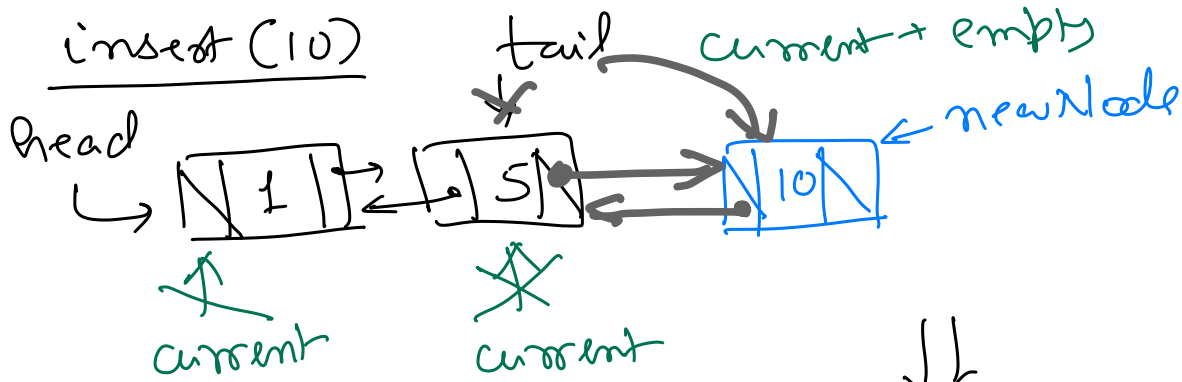
- Make the current node's previous node come before newNode. // Set newNode's previous to current node's previous.

- Make newNode come after the current node's previous node. // Set current node's previous node's next to newNode.

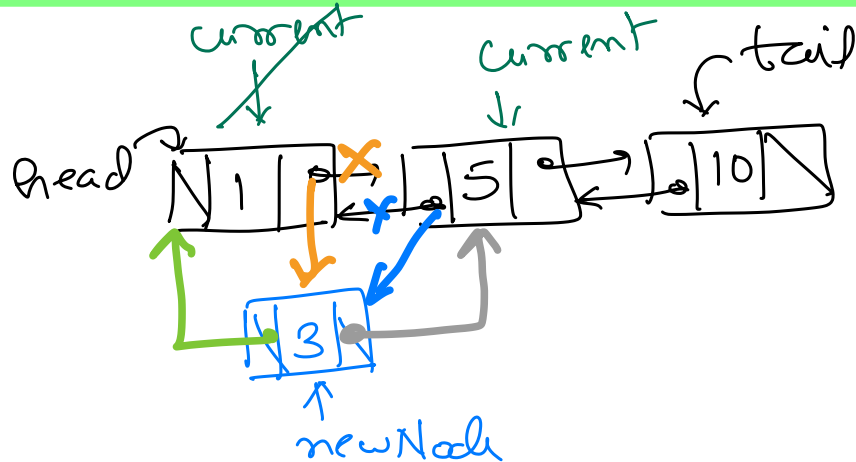
- Make newNode come before the current node. // Set current node's previous to newNode.

- Stop.

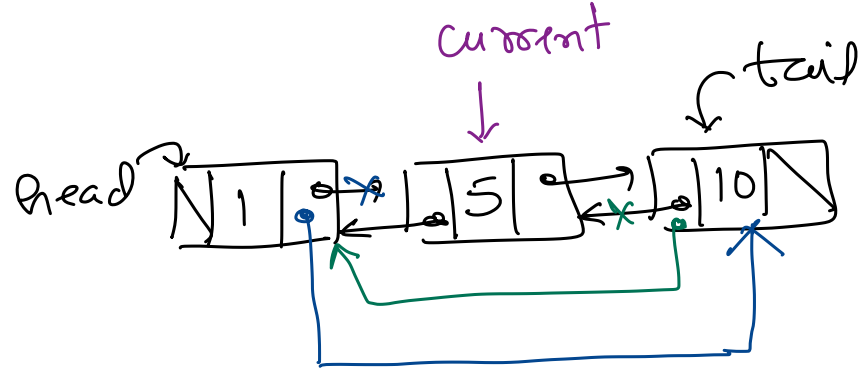
insert(10)



insert(3)



Delete node from doubly list



delete(5)

- ① Set current node's previous node's next to current node's next.
- ② Set current node's next node's previous to current node's previous.

Special cases

- ① Empty list
- ② Element not found
- ③ Delete first element
- ④ Delete last element.
- ⑤ Delete the only element in list.

Delete (element)

// Find the node to be deleted - current node

- Set current to first node (head)
- while (current is not empty) do
 - if (current node's data = element) then
 - // Found the node - end the traversal.
 - End the traversal.
 - Move current to current's next node

// Have we found the node to be deleted?

- if (current is empty) then
 - Stop.

// Delete first node?

- if (current is first node) then
 - Move head to head's next node.

// Has the list become empty => list has only 1 node

- if (head is empty) then
 - Set tail to empty.

Else

- Set the previous of head to empty.
- Release memory of the current node. (Not required for JAVA).

- Stop.

// Delete last node?

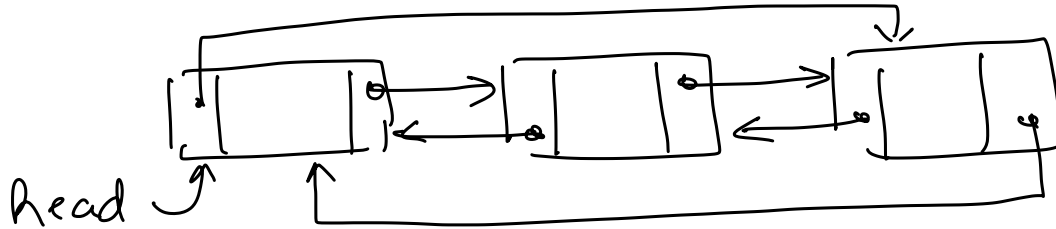
- if (current is last node) then
 - Move tail to tail's previous node.
 - Set the next of tail node to empty.
 - Release memory of the current node. (Not required for JAVA).
 - Stop.
- Make current's next node come after current's previous node. // **Set current node's previous node's next to current node's next node.**
- Make the current node's previous node come before the current node's next node. // **Set current's next node's previous to current's previous node.**
- Release memory of the current node. (Not required for JAVA).
- Stop.

Circular Doubly Linked List

How to implement it?

Issue if first and last nodes are connected to form a cycle.

How will we do the traversal?

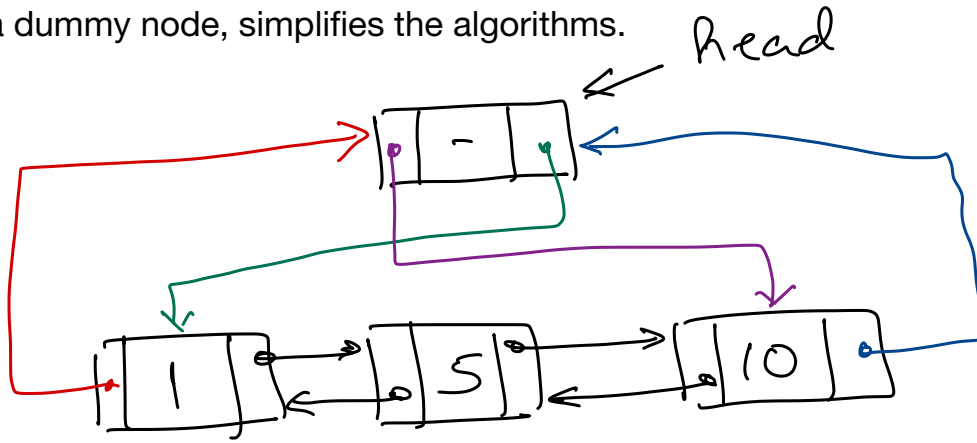


head $\rightarrow$ empty

Traversal

```
current = head;  
while (current != head) {  
    ...  
    current = current->next;  
}
```

How using a dummy node, simplifies the algorithms.



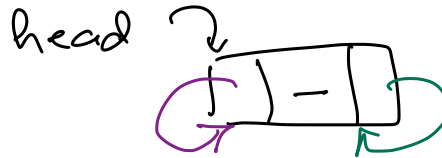
fwd traversal

```
current = head.next;  
while (current != head) {  
    :  
}
```

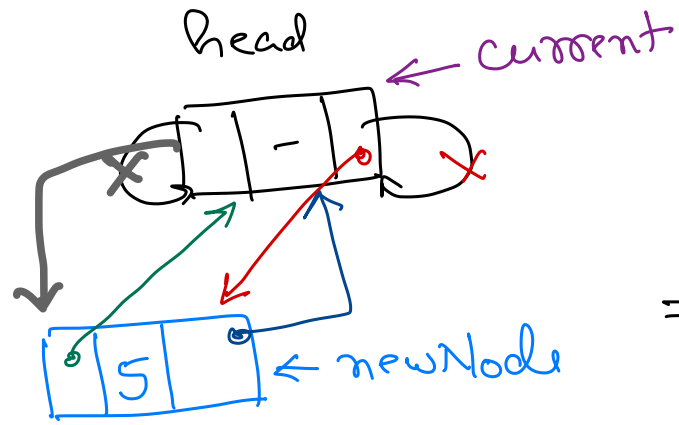
rev traversal

```
current = head.previous;  
while (current != head) {  
    :  
}
```

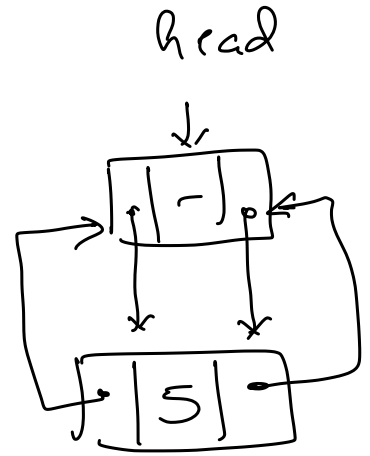
Empty list



insert (5)



$\Rightarrow$



Insert(element)

// 1. Create new node

- Make memory for new element, say newNode.
- Store element in newNode's data.
- Set newNode's next and previous to empty.

// 3. Traverse list to find node - current node.

- Set current to head (first node).
- while (current is not empty) do
 - if (current node's data > element) then
 - // Found the node, end the traversal.
 - End the traversal.
- Set current to current's next node.

// 6. Add a new node between current and current's previous node.

- Make the current node come after newNode. // **Set newNode's next to current.**
- Make the current node's previous node come before newNode. // **Set newNode's previous to current node's previous.**
- Make newNode come after the current node's previous node. // **Set current node's previous node's next to newNode.**
- Make newNode come before the current node. // **Set current node's previous to newNode.**
- Stop.

Delete (element)

// Find the node to be deleted - current node

- Set current to first node (head)
- while (current is not empty) do
 - if (current node's data = element) then
 - // Found the node - end the traversal.
 - End the traversal.
- Move current to current's next node

// Have we found the node to be deleted?

- if (current is empty) then
 - Stop.

- Make current's next node come after current's previous node. // Set current node's previous node's next to current node's next node.
- Make the current node's previous node come before the current node's next node. // Set current's next node's previous to current's previous node.
- Release memory of the current node. (Not required for JAVA).
- Stop.